

Fondamenti di Analisi dei Dati

from **data analysis** to **predictive techniques**

Prof. Antonino Furnari (antonino.furnari@unict.it)

Corso di Studi in Informatica

Dip. di Matematica e Informatica

Università di Catania



Università
di Catania

Density Estimation

Estimating where the data lives: histograms, kernels, Gaussians and mixtures of them

What is Density Estimation?

Clustering algorithms mine the structure of data by breaking it into coherent groups. An alternative, **smoother** approach is to study the probability

$$P(X)$$

where X is the data. If we could give a probability value to any point in space, clusters would follow naturally as the **zones with high probability**.

In practice, when X is continuous, we aim to find the density function $f(x)$. Knowing the distribution of the data is useful for a number of things:

Exploratory Data Analysis

Understand the shape, modality and skewness of a distribution, for a deeper reading of the underlying patterns.

Anomaly Detection

Identify unusual points or events by modelling typical behaviour — useful in fraud detection and quality control.

Clustering and Classification

Discover high-density regions to group similar points, and inform the decision boundaries of a classifier.

Generative Models

Learn a complex distribution well enough to synthesise new, realistic samples from it.



Two Approaches to Density Estimation

Non-Parametric Methods

Estimate the density directly from the data, without strong assumptions on the distribution that generated it.

Advantages: few hyper-parameters, no assumptions, flexible.

Disadvantages: no analytical form for $f(x)$, computationally intensive.

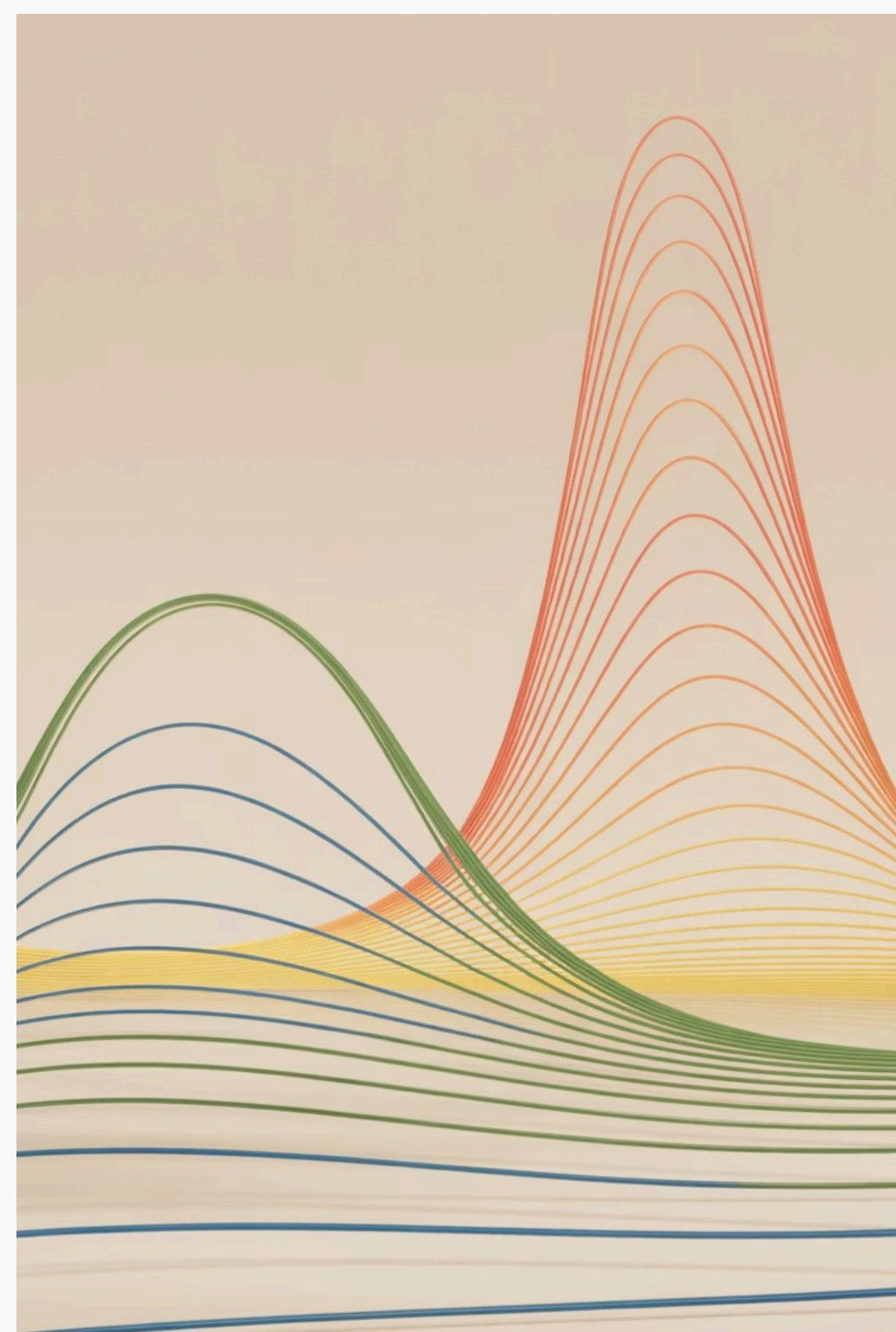
The two families answer the same question with opposite trades: the first keeps the data and asks it every time, the second summarises it into a handful of numbers.

Parametric Methods

Fit a known distribution — a Gaussian, say — to the data.

Advantages: analytical form, compact representation, interpretable.

Disadvantages: strong assumptions, requires optimisation, may not fit complex data.



Non-Parametric Approaches

We will see two kinds of non-parametric approaches.

1

Fixed Windows

Discretise the feature space into a fixed grid of bins and count the points in each — the multidimensional histogram.

2

Mobile Windows

Centre a window on every point where we want the density, and sum the influence of the data — kernel density estimation.

Both estimate the density directly from the data, but mobile windows produce markedly smoother and more accurate results.



Fixed Windows: The Histogram Method

The simplest approach extends the 1-D histogram to several dimensions, with a grid of bins (regions) R_i . For f to be a valid density it must satisfy

$$\int f(\mathbf{x}) d\mathbf{x} = 1$$

so we define the density of a bin as the probability of the bin — the number of elements in it — divided by the **volume** of the bin:

$$f(\mathbf{x} \in R_i) = \frac{P(R_i)}{V(R_i)} = \frac{|R_i|/N}{V(R_i)}$$

If the bin is a 2×2 cell, its "volume" is 4. This is coherent with physics, where density is mass over volume, $\rho = \frac{m}{V}$.

The normalisation makes the integral equal to 1:

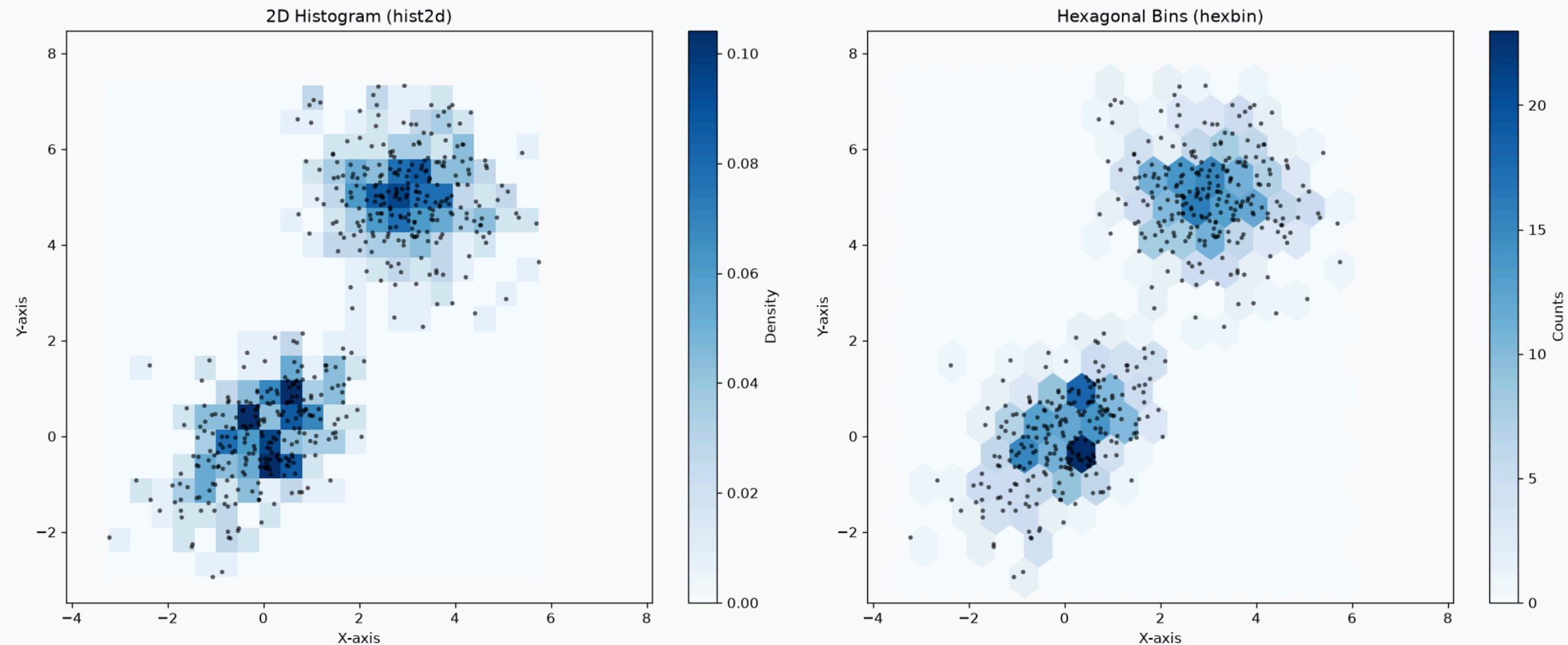
$$\int f(\mathbf{x}) d\mathbf{x} = P(\mathbf{x} \in \cup_i R_i) = \sum_i P(R_i) = \sum_i f(\mathbf{x} \in R_i) V(R_i) = \sum_i \frac{|R_i|/N}{V(R_i)} V(R_i) = 1$$

It also means that a small, crowded bin has high density while a large, sparse bin has low density, even with the same number of points in them.



Histogram Variants: Square vs. Hexagonal Bins

Fixed Window Density Estimation



Hexagonal binning tiles the plane more naturally than a square grid — every neighbour of a hexagon is at the same distance — which often produces a smoother visual result for the same number of points.



Limitations of Histograms

Arbitrary boundaries

Bin edges are placed without regard to the structure of the data, so the same dataset can produce different densities depending only on where the grid starts.

Blocky results

The estimate is discontinuous and "staircase-like", which fails to capture the smooth nature of a true probability density.

The curse of dimensionality

The method generalises to N dimensions on paper. In practice the number of bins grows exponentially and almost all of them are empty.

These limitations motivate a more sophisticated approach: **mobile windows**, which adapt to the data instead of imposing a rigid grid.



Kernel Density Estimation (KDE)

KDE solves the problems of the fixed window by placing a "window" at the point where we want the density and asking how much data falls inside it. The mobile window approach, also called the **Parzen window** method, produces smooth and flexible estimates.

01

Center a window

Place a window of radius h at the point $\mathbf{x}$.

02

Count nearby points

Compute $N(\mathbf{x}, h)$, the number of data points inside the window.

03

Compute density

Density = probability in the window, divided by the volume of the window.



The Naive Circular Neighborhood

The circular kernel centres a hypersphere of radius h at $\mathbf{x}$ and counts the points inside:

$$f(\mathbf{x}) = \frac{|N(\mathbf{x}, h)|/N}{V(h)}$$

where

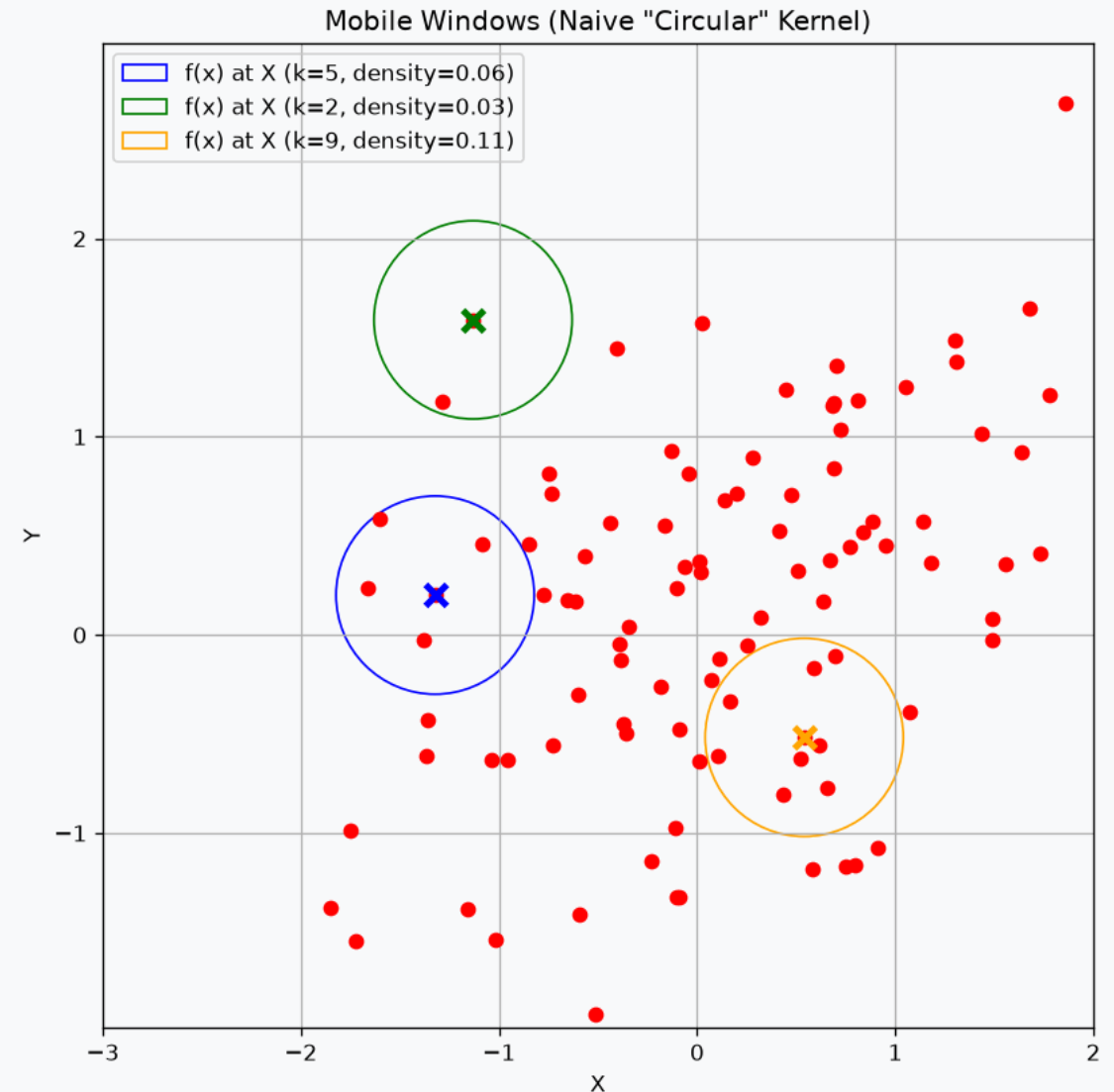
$$N(\mathbf{x}, h) = \{\mathbf{x}' \in \mathbf{X} \text{ s.t. } \|\mathbf{x}' - \mathbf{x}\|_2 \leq h\}$$

is the neighbourhood centred at $\mathbf{x}$ with radius h , and $V(h)$ is the volume of that hypersphere — in 2-D, $V(h) = \pi h^2$. It can be shown that

$$\int_{\mathbf{x} \in \mathbb{R}^d} f(\mathbf{x}) d\mathbf{x} = 1$$

The problem: hard decisions create sensitivity. A point is either 100% in or 100% out, so moving $\mathbf{x}$ slightly can make the density jump.

This binary decision produces non-smooth, bumpy estimates. We need kernels that assign a weight which *decreases* as distance grows.



Kernel View

We can express density estimation as a sum over all data points, each weighted by a **kernel function** K_h :

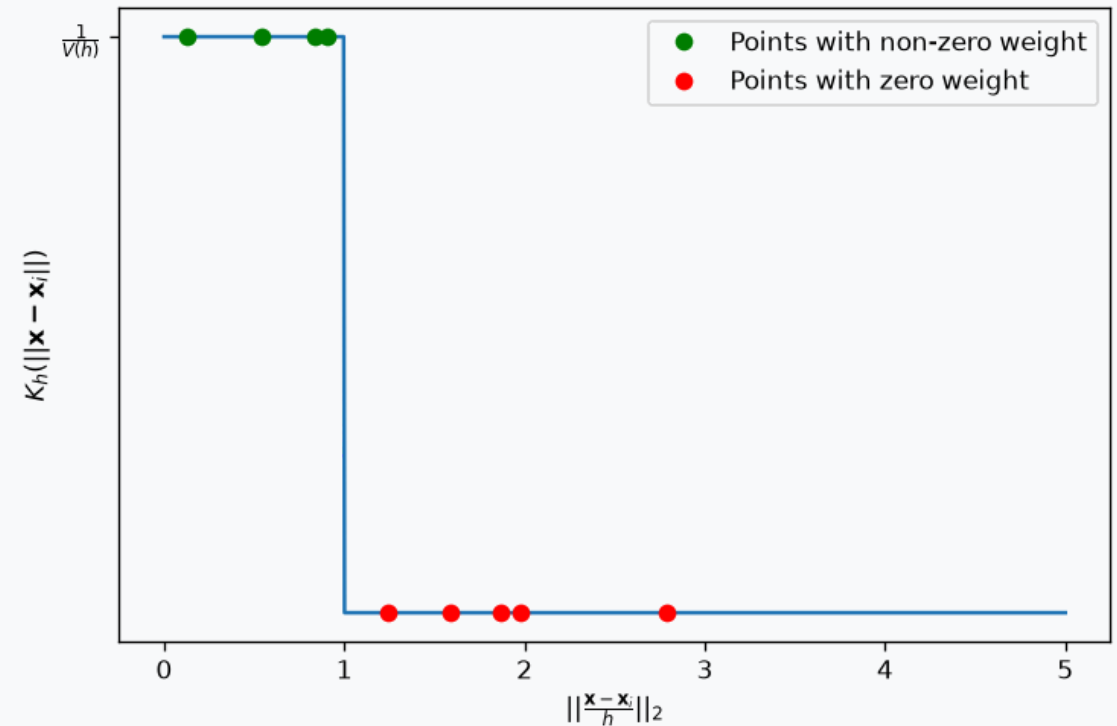
$$f(\mathbf{x}) = \frac{1}{|\mathbf{X}|} \sum_{i=1}^{|\mathbf{X}|} K_h(\mathbf{x}_i - \mathbf{x})$$

A kernel function takes the difference of two points and assigns it a weight, saying how much that point contributes to the density at $\mathbf{x}$. We expect points closer to $\mathbf{x}$ to contribute more.

The circular kernel makes hard decisions, assigning the uniform weight $\frac{1}{V(h)}$ inside the radius and zero outside:

$$K_h(\mathbf{x}_i - \mathbf{x}) = \begin{cases} \frac{1}{V(h)} & \text{if } \left\| \frac{\mathbf{x}_i - \mathbf{x}}{h} \right\|_2 \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

In the plot, points at nearly identical distances — one just inside h , one just outside — get vastly different weights. Small shifts of $\mathbf{x}$ turn green points (non-zero weight) into red ones (zero weight), which makes the estimate unstable.



The Solution: Smooth Kernels

Smooth kernels remove the hard boundary by assigning weights that decrease gradually with distance. The **Gaussian kernel** is the popular choice:

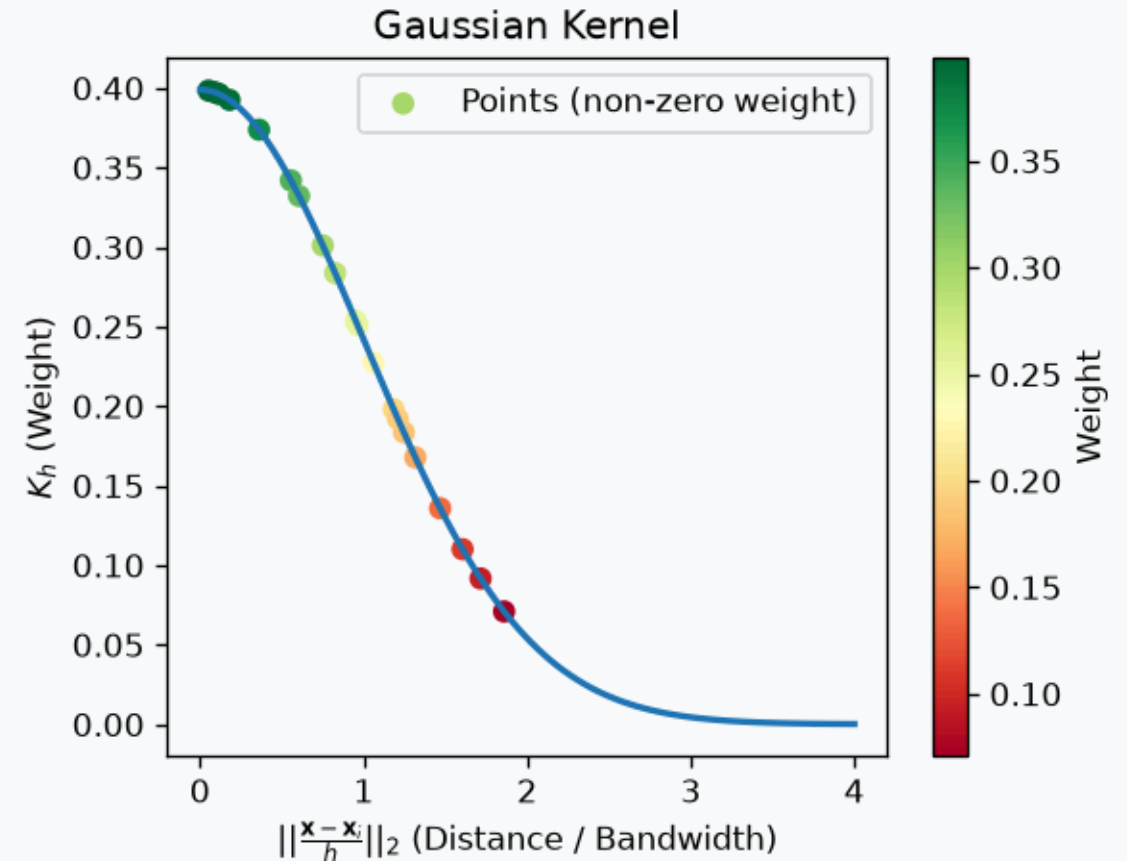
$$K_h(\mathbf{x}_i - \mathbf{x}) \propto \exp\left(-\frac{|\mathbf{x}_i - \mathbf{x}|^2}{2h^2}\right)$$

With a smooth kernel *all* points contribute to the density at $\mathbf{x}$, but nearby points contribute far more than distant ones.

The general KDE formula is still a sum of kernels, one centred at each data point:

$$f(\mathbf{x}) = \frac{1}{N} \sum_{i=1}^N K_h(\mathbf{x}_i - \mathbf{x})$$

where $N = |\mathbf{X}|$. This produces a continuous estimate of the density.



When Efficiency Meets Smoothness: The Epanechnikov Kernel

The Gaussian kernel solves the sensitivity problem, but **it requires computing an exponential for every data point, at every point where we want the density**. On a large dataset this becomes prohibitive.

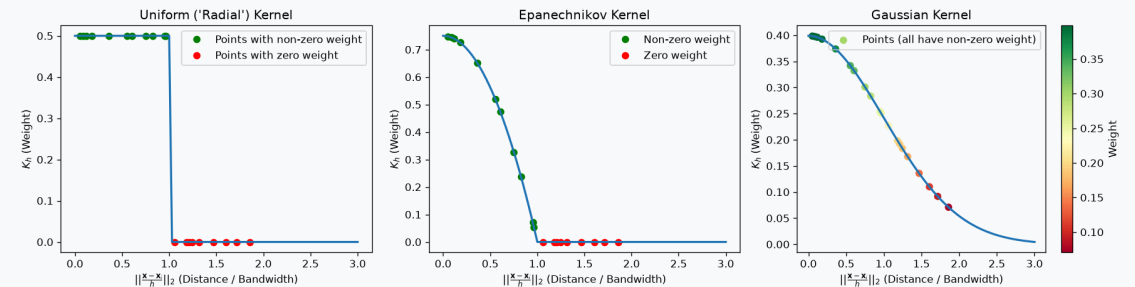
The **Epanechnikov kernel** is the compromise: smooth, and zero beyond distance h , which cuts the computation down to the neighbours.

$$K_h(\mathbf{x}_i - \mathbf{x}) = \frac{3}{4h^2} \left(1 - \frac{|\mathbf{x}_i - \mathbf{x}|^2}{h^2} \right) \mathbb{I}(|\mathbf{x}_i - \mathbf{x}| \leq h)$$

where $\mathbb{I}$ is the indicator function:

$$\mathbb{I}(x) = \begin{cases} 1 & \text{if } x \text{ is true} \\ 0 & \text{otherwise} \end{cases}$$

The quadratic profile keeps the estimate smooth, while the compact support means only nearby points are ever considered. It is also optimal in the sense of minimising the mean integrated squared error.

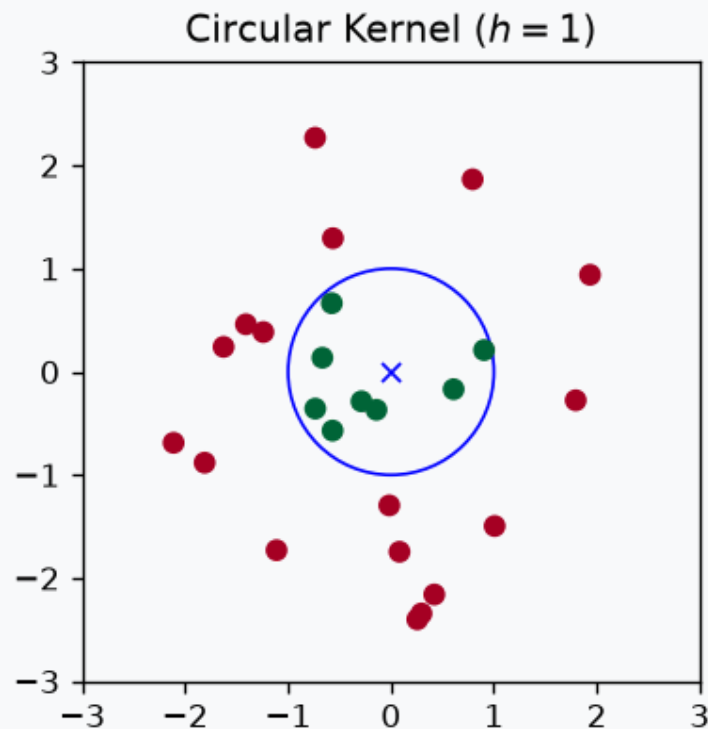


Kernel Comparison

01

Uniform (circular) kernel

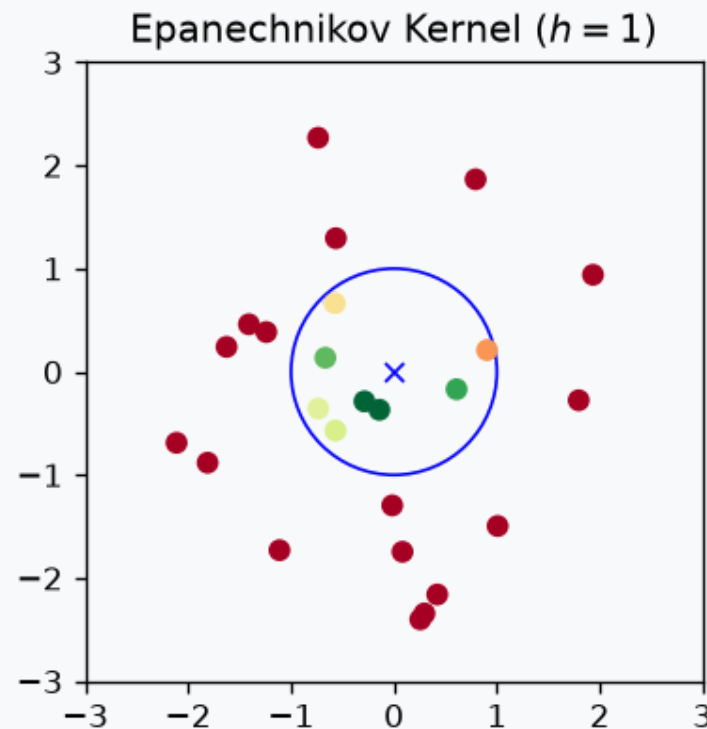
Binary weighting, and visible boundaries. Points inside radius h weigh the same, points outside weigh nothing: the estimate has artificial discontinuities.



02

Epanechnikov kernel

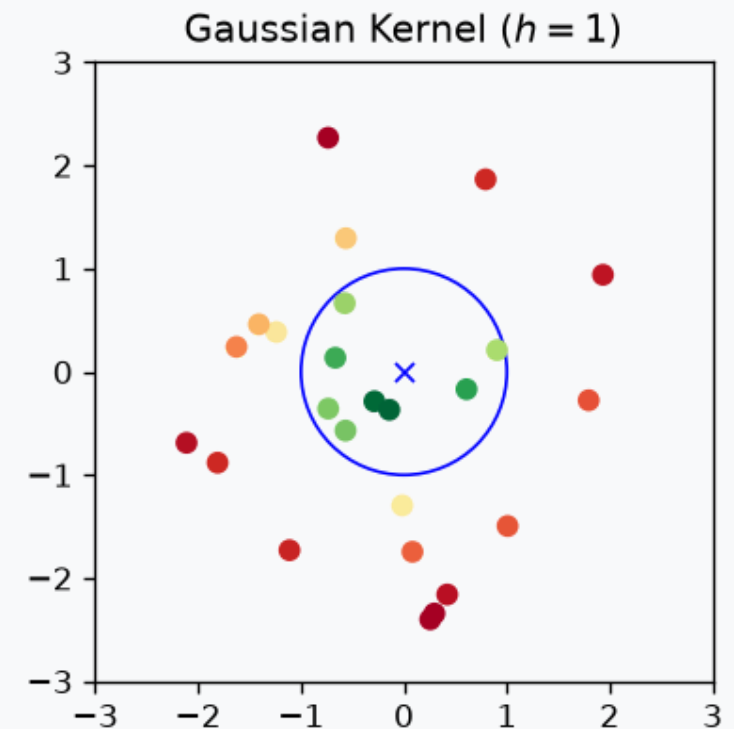
Quadratic decay gives smoothness, compact support limits the computation. The weight falls parabolically from the centre to h , then drops to zero.



03

Gaussian kernel

Infinite support with exponential decay. Every point receives a non-zero weight, which gives the smoothest estimates, at the highest cost.



The same twenty points, and the weight each one receives under the three kernels at $h = 1$: a hard cut, a parabola that reaches zero at the circle, and a tail that never quite does.



The Bandwidth Dilemma: Bias-Variance Tradeoff

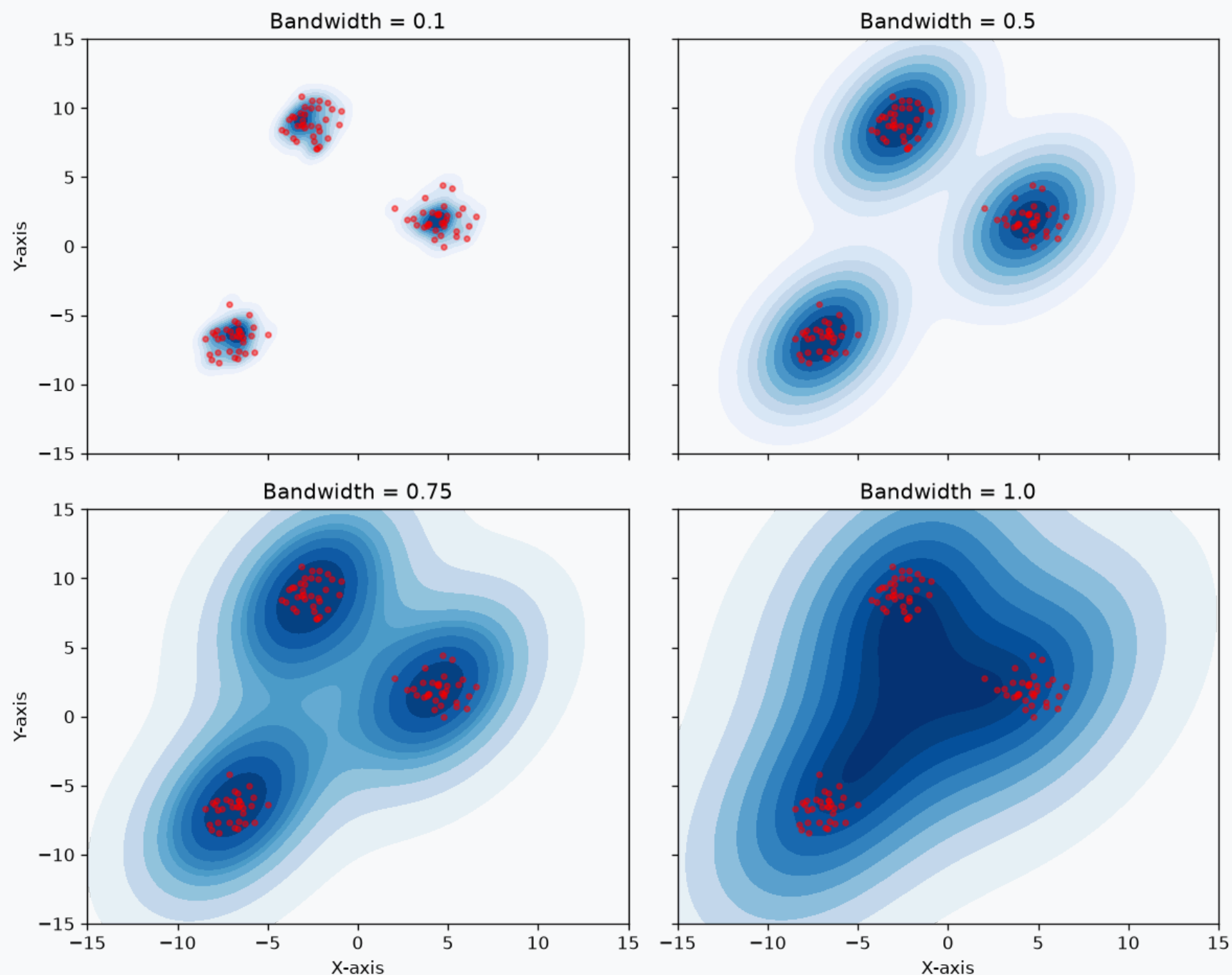
The bandwidth h is **the single most important choice** in density estimation — kernel *shape* matters far less than getting h right. It controls the bias-variance tradeoff:

Small bandwidth ($h = 0.1$)

- Low bias, high variance
- trusts only a tiny local region
- spiky, noisy estimates
- **overfits** the training data
- reveals individual points, misses the structure

Large bandwidth ($h = 1.0$)

- High bias, low variance
- averages over a huge area
- over-smoothed estimates
- **underfits** the data
- hides the true structure, such as separate clusters



The sweet spot — h between 0.5 and 0.75 here — balances local fidelity with global smoothness: three blobs, and no spikes.

KDE in Action

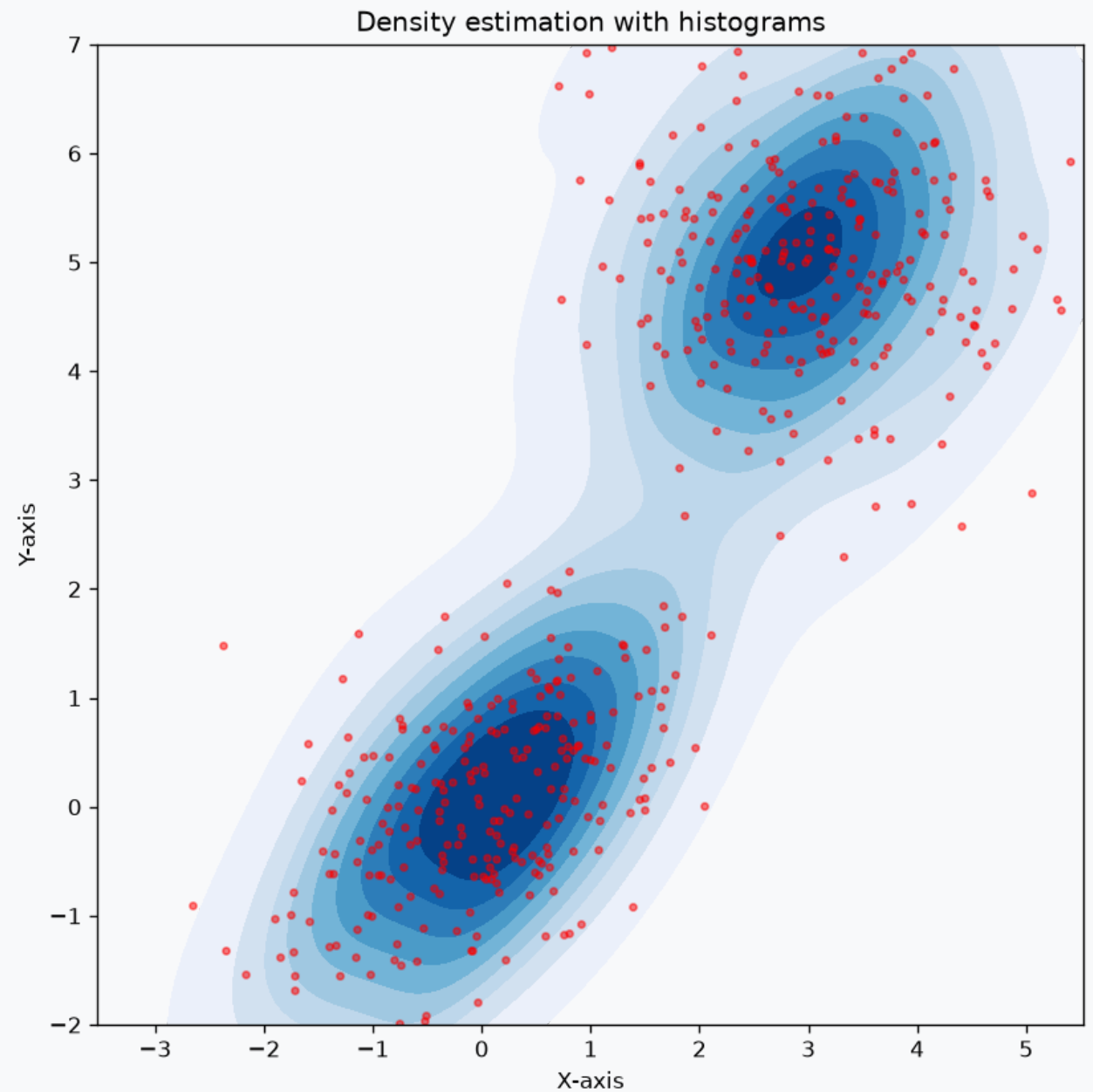
Using KDE in Python is simple. This example uses `seaborn`, but similar plots come directly from `pandas` or from dedicated `scipy` functions.

```
import matplotlib.pyplot as plt
import seaborn as sns

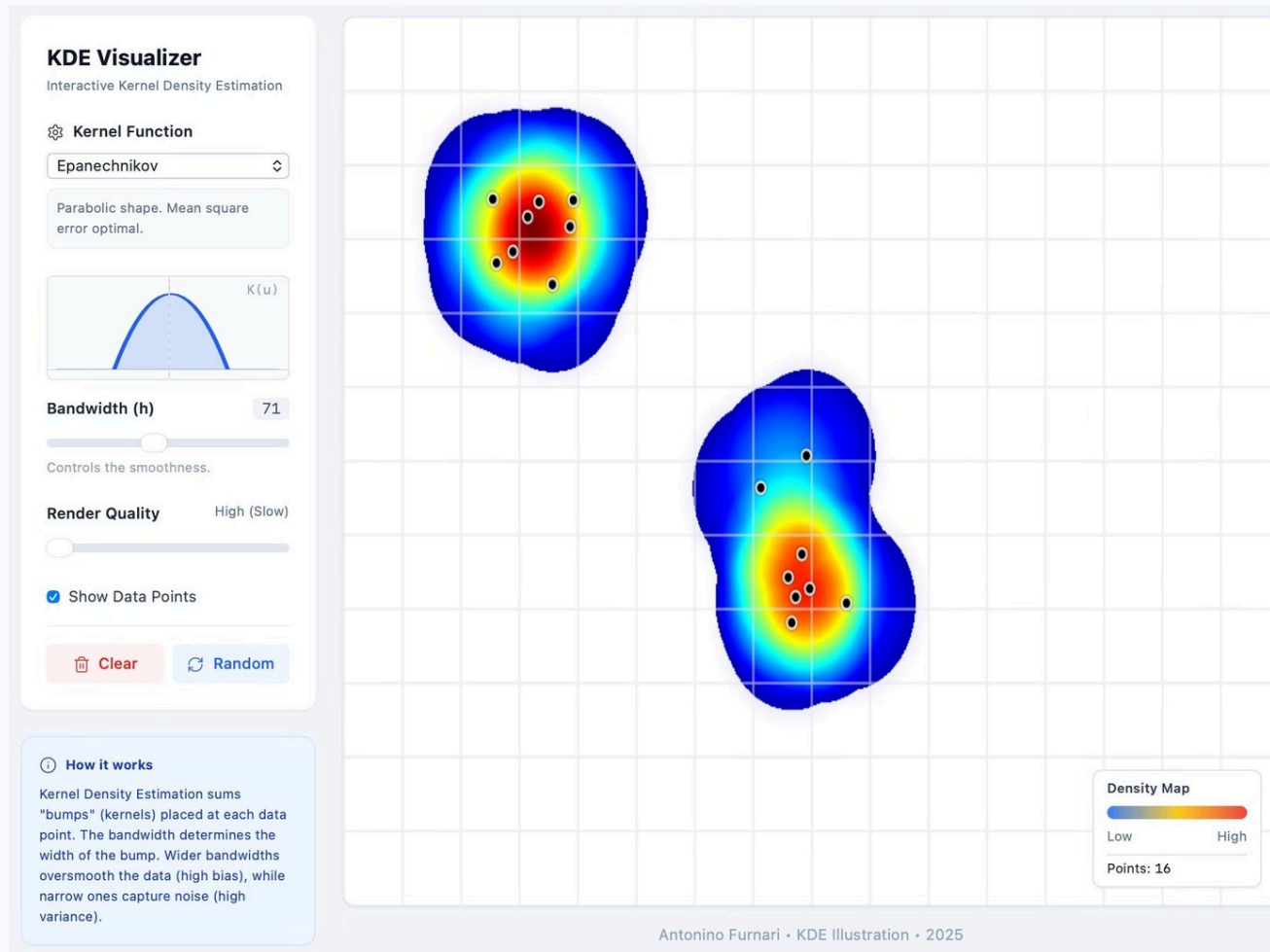
sns.kdeplot(x=data[:, 0],
            y=data[:, 1],
            cmap='Blues',
            fill=True)

plt.show()
```

With a good kernel and a well-chosen bandwidth, we get a smooth, accurate, non-parametric estimate of the density — and `kdeplot` does all of it for us.



Kernel Density Estimation: Live Demo



An interactive visualiser: click to place points, choose the kernel, drag the bandwidth and watch the estimate change.

Open the KDE live demo

The thing to try in class: place a handful of points, set a tiny bandwidth — every point becomes its own island — then raise it until the islands merge into one blob. Somewhere in between is the estimate we want.



Parametric Density Estimation: Philosophy and Tradeoffs

Advantages

- **Analytical formulation:** known equations allow mathematical reasoning and optimisation.
- **Compact representation:** store the parameters, not the whole dataset.
- **Interpretability:** the parameters say something about the data — the mean is the point of highest density.
- **Data efficiency:** strong assumptions work better when data is limited.

Disadvantages

- **Optimization required:** finding the parameters can be expensive.
- **Strong assumptions:** the model has to match the true distribution.
- **Potential mismatch:** if the assumptions are violated, the estimate is simply wrong.
- **Limited flexibility:** a fixed family cannot capture an arbitrary distribution.



Fitting a Gaussian: Maximum Likelihood Estimation

The most common parametric method assumes the data follows a **multivariate Gaussian distribution**, defined entirely by a mean vector μ and a covariance matrix Σ :

$$N(\mathbf{x}; \mu, \Sigma) = \frac{1}{\sqrt{(2\pi)^d \det(\Sigma)}} e^{-\frac{1}{2}(\mathbf{x}-\mu)^T \Sigma^{-1}(\mathbf{x}-\mu)}$$

Maximum Likelihood Estimation (MLE) finds the parameters that maximise the probability the model assigns to the observed data. The likelihood is

$$P(\mathbf{X}|\mu, \Sigma) = \prod_i \mathcal{N}(\mathbf{x}_i|\mu, \Sigma)$$

assuming the examples are independent and identically distributed. Since products of small numbers are awkward, we work with the **log-likelihood**:

$$\log P(\mathbf{X}|\mu, \Sigma) = \sum_{i=1}^N \log \mathcal{N}(\mathbf{x}_i|\mu, \Sigma)$$



MLE Solution: Elegantly Simple

Remarkably, the optimal parameters have closed-form solutions — the sample mean and the sample covariance matrix.

The sample mean

The ML estimate for μ is the average of all data points:

$$\mu_{ML} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i$$

The "most likely" centre is exactly where the data is centred on average.

The bottom line: "fitting a Gaussian" is computing a mean and a covariance. What sounds like complex statistical inference reduces to basic descriptive statistics — the beauty is in the justification, not in the arithmetic: these estimates are the ones that maximise the likelihood of what we observed.

The sample covariance

The ML estimate for Σ captures the spread and the correlation structure:

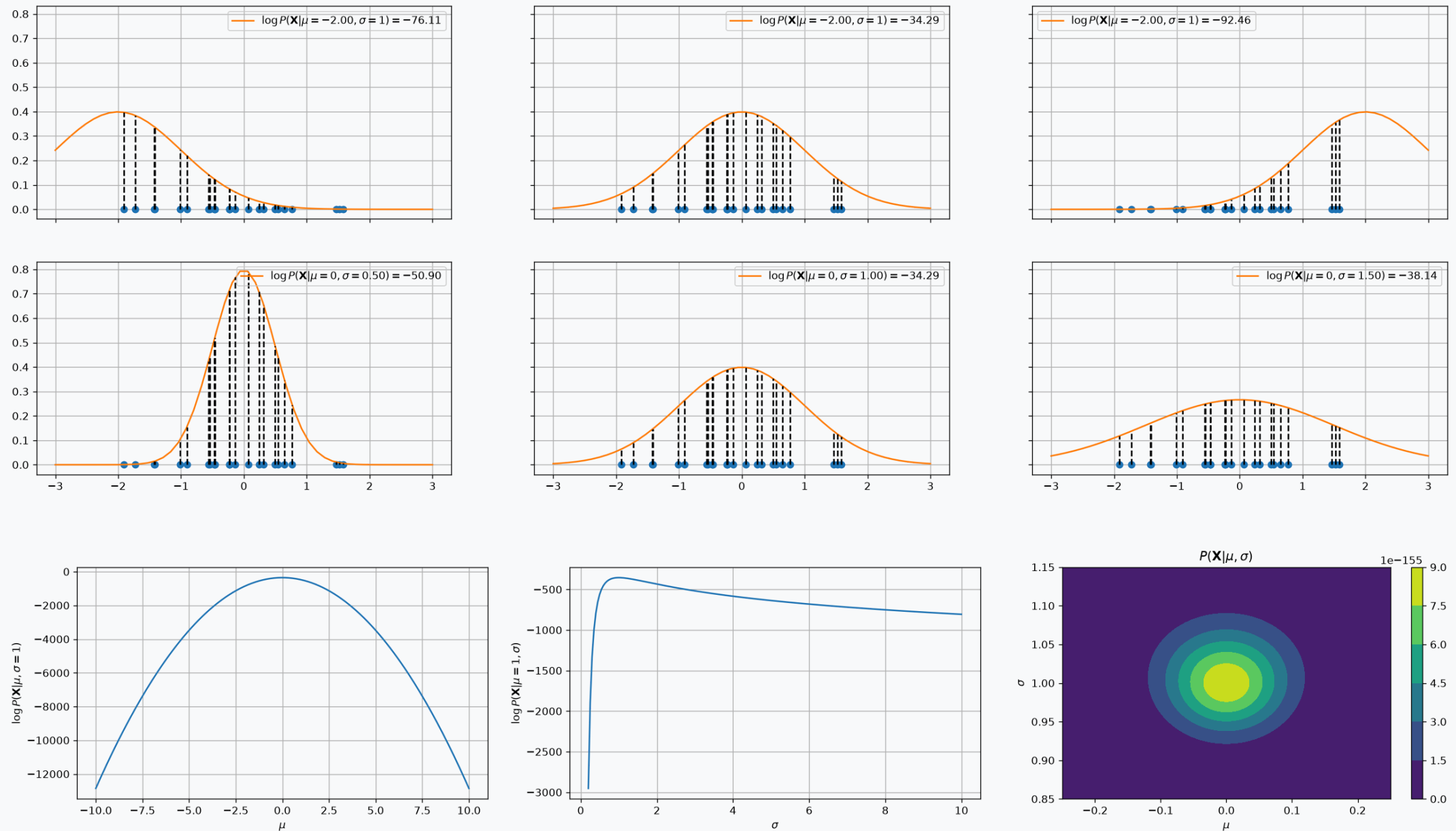
$$\Sigma_{ML} = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i - \mu_{ML})(\mathbf{x}_i - \mu_{ML})^T$$

It measures how the data varies around the mean, in every direction.



How Does Optimization Work?

The plots below show the cost function — the negative log-likelihood — for different choices of the parameters. Closed-form solutions exist for the Gaussian; in other cases one uses iterative optimisation, as with the logistic regressor.



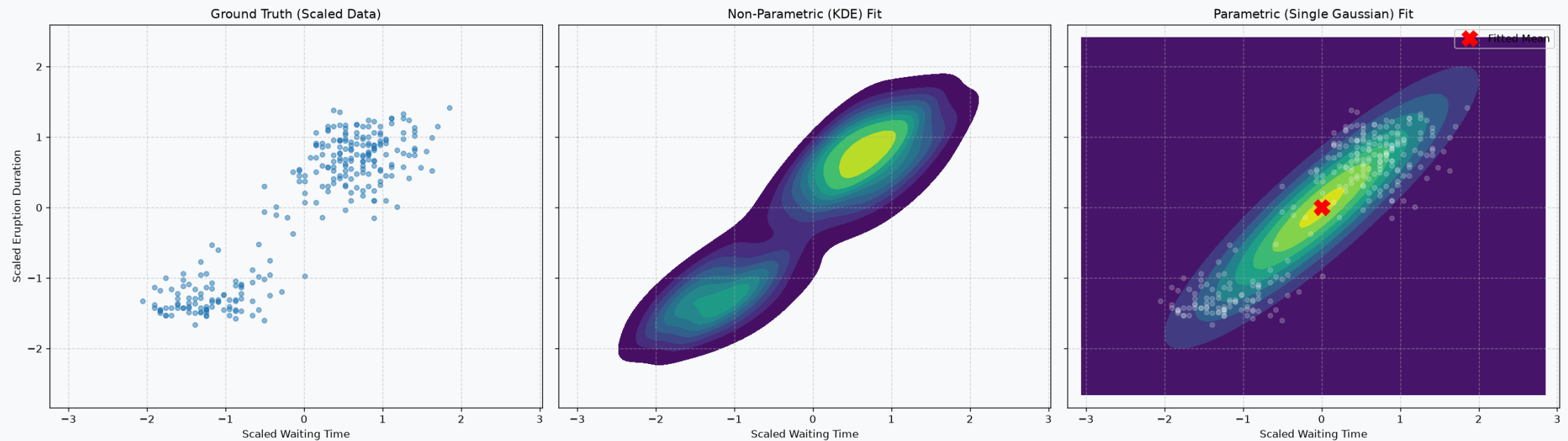
The first row moves the mean, the second changes the width. The curves below show the log-likelihood as a function of μ alone, of σ alone, and the joint surface: one clean maximum, exactly where the sample mean and the sample variance sit.



When Single Gaussians Fail

Single Gaussian models assume **unimodal** data — one cluster. This fails on many real datasets with several distinct groups, such as the Old Faithful geyser data with its two clear clusters.

Model Bake-Off: Unimodal vs. Multimodal Data



Fitting a single Gaussian to bimodal data places the mean in the empty space *between* the clusters, missing the structure entirely. Non-parametric KDE finds both modes correctly, but gives no compact mathematical model.

- **The question:** can we build a parametric model flexible enough to handle multi-modal data?

Gaussian Mixture Models (GMM)

A **Gaussian Mixture Model** answers it by modelling the data as a weighted average — a "mixture" — of K Gaussian components:

$$P(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \mu_k, \Sigma_k)$$

Mixing coefficients (π_k)

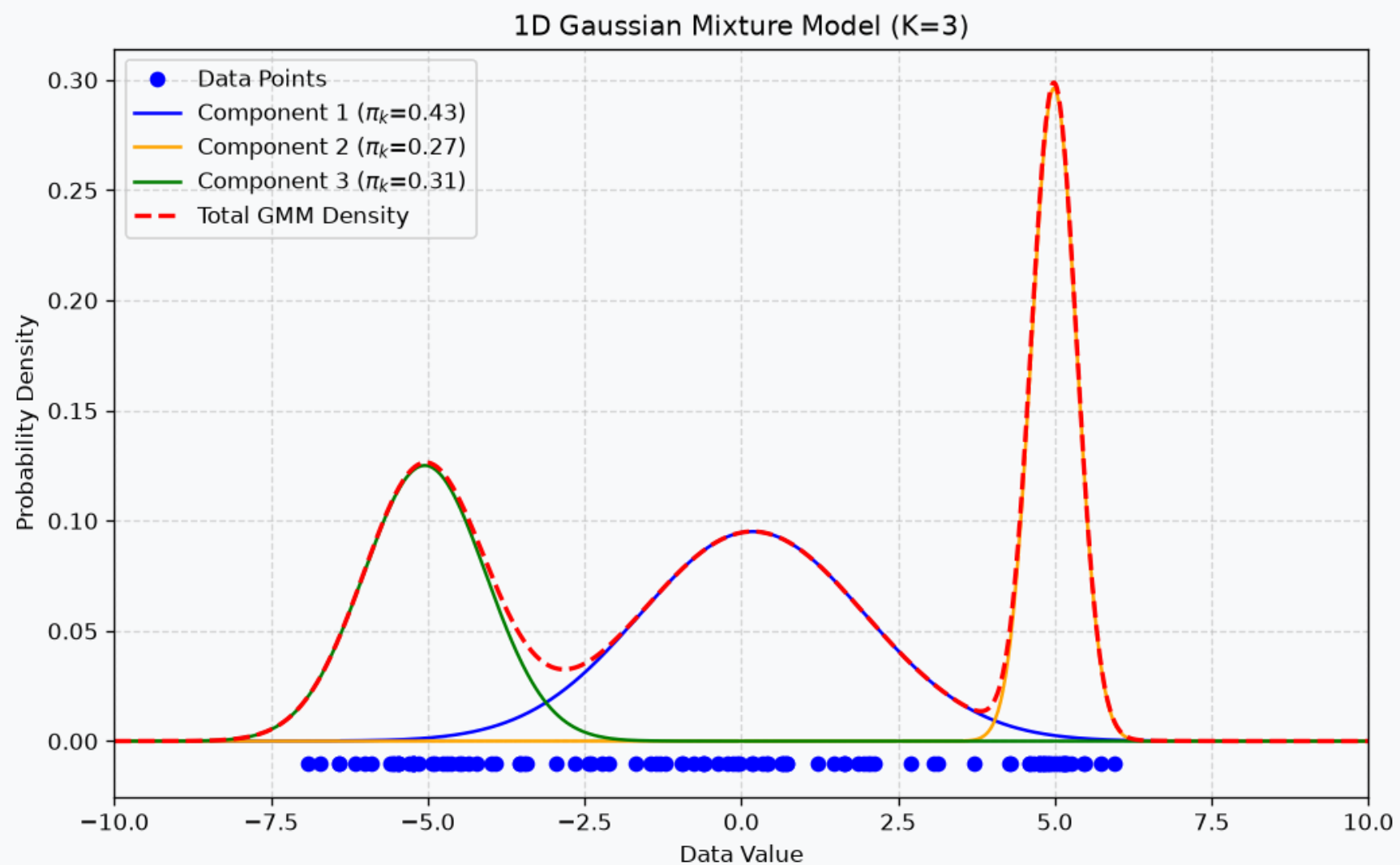
The weight, or prior probability, of each component. They must sum to 1.

Means (μ_k)

The centre, or location, of each of the K Gaussian components.

Covariances (Σ_k)

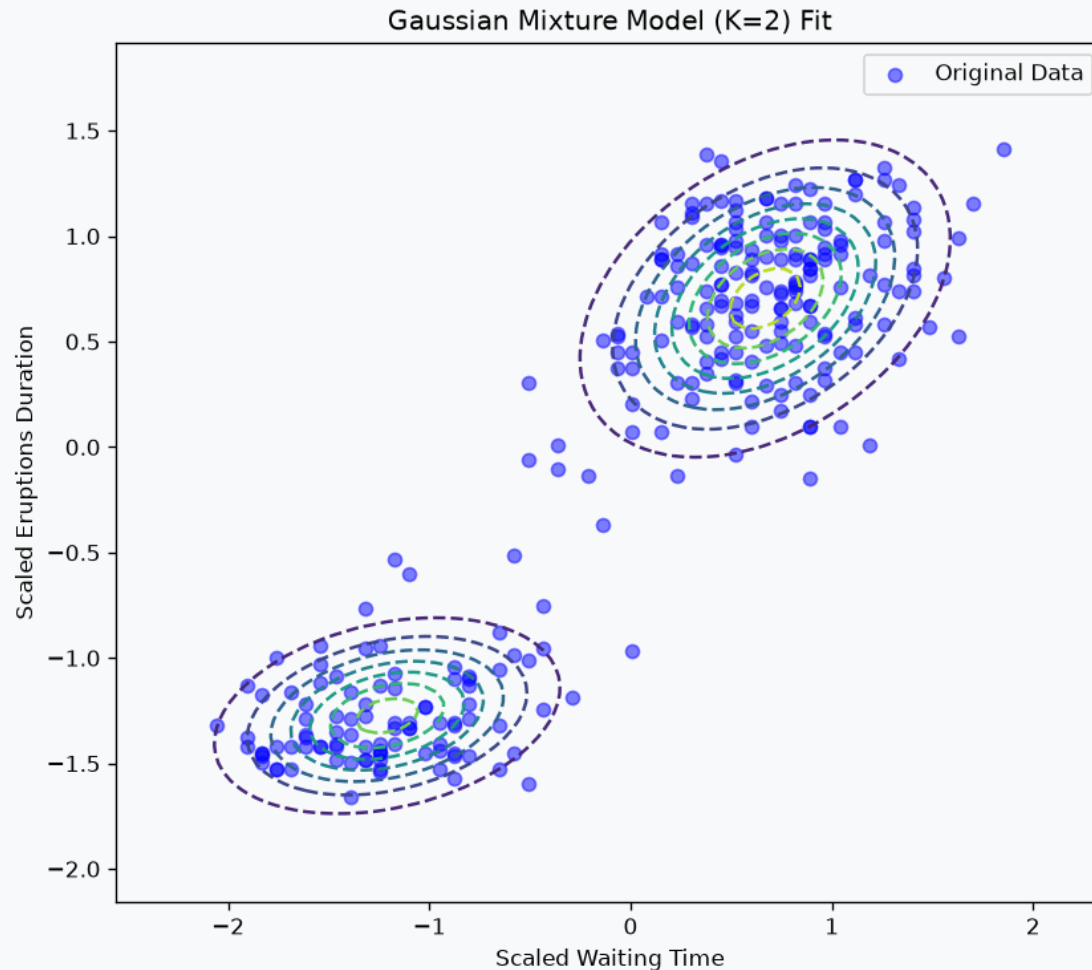
The shape, spread and orientation of each of the K components.



Three components with weights 0.43, 0.27 and 0.31, and the density they sum to: two broad modes and one narrow spike, which no single Gaussian could produce.



GMM Example on Geyser Data



The same geyser data that defeated the single Gaussian, fitted with $K = 2$ components.

Each component has its own mean and its own covariance, so the model puts one tilted ellipse on the short-eruption group and another on the long-eruption group, instead of one ellipse stretched across the gap.

We now have both things at once: the flexibility to follow the shape of the data, and a compact parametric model — here, two means, two covariance matrices and two weights.



GMM as "Soft Clustering"

GMM offers a nuanced alternative to the hard assignments of K-means. Instead of placing each point rigidly in one cluster, it computes the probability that a point belongs to each cluster: a **soft assignment**.

The latent variable Z

We introduce a **latent** (hidden) variable Z , where $Z = k$ means that an example $\mathbf{x}$ was generated by cluster k . We never observe Z directly, but it is what makes the model a story about how the data came to be.

Computing soft assignments with Bayes' theorem

We want the posterior probability $P(Z = k|\mathbf{x})$, which Bayes' theorem gives us:

$$P(Z = k|\mathbf{x}) = \frac{P(\mathbf{x}|Z = k)P(Z = k)}{P(\mathbf{x})}$$

$P(\mathbf{x}|Z = k)$ — the likelihood

The probability of observing $\mathbf{x}$ if it came from cluster k : the value of the k -th Gaussian component, $\mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k)$.

$P(Z = k)$ — the prior

How common cluster k is in general: the mixing coefficient π_k .

$P(\mathbf{x})$ — the evidence

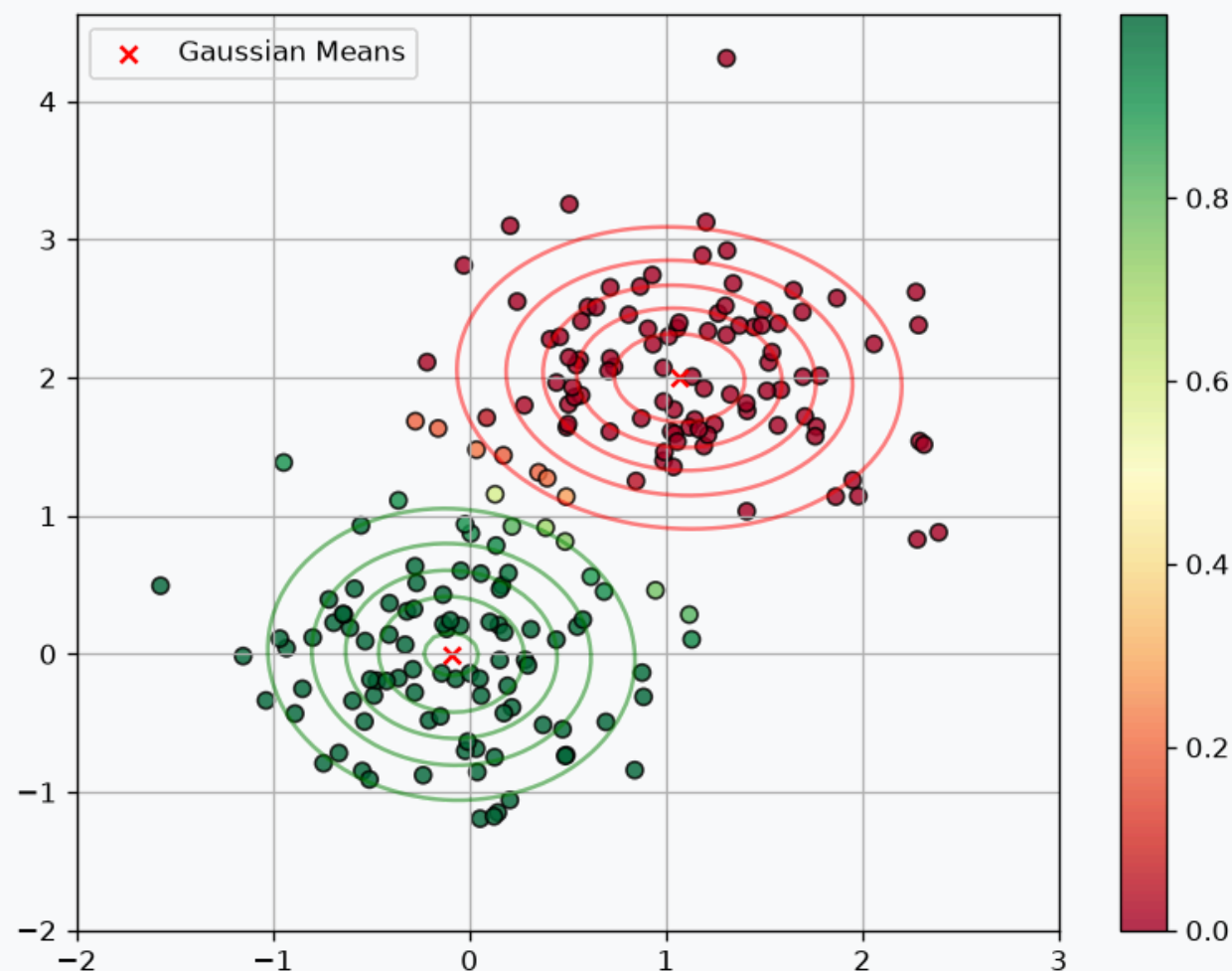
The total probability of observing $\mathbf{x}$ across all clusters: the sum over the components, $\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}|\mu_j, \Sigma_j)$.



GMM as "Soft Clustering": Responsibilities

Combining the three terms, the soft assignment — the **responsibility** γ_k — of cluster k for point $\mathbf{x}$ is:

$$\gamma_k = P(Z = k|\mathbf{x}) = \frac{\pi_k \mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}|\mu_j, \Sigma_j)}$$



The plot shows the points coloured by their responsibility towards one of the two components: deep green for points that belong to it almost surely, deep red for points that almost surely belong to the other.

The interesting colours are the ones in between, along the line where the two components meet. K-means has to give those points a cluster; a GMM can say **0.55** and **0.45**, which is both more honest and more useful downstream.

How to "Learn" a GMM: Optimization by Maximum Likelihood

A Gaussian Mixture Model is a weighted sum of K Gaussian components:

$$P(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \mu_k, \Sigma_k)$$

To define it fully we need the mixing coefficients π_k , the means μ_k and the covariances Σ_k of each component. As with a single Gaussian, we use **maximum likelihood**: find the parameters $\theta = (\pi, \mu, \Sigma)$ maximising the log-likelihood of the observed data $\mathbf{X}$:

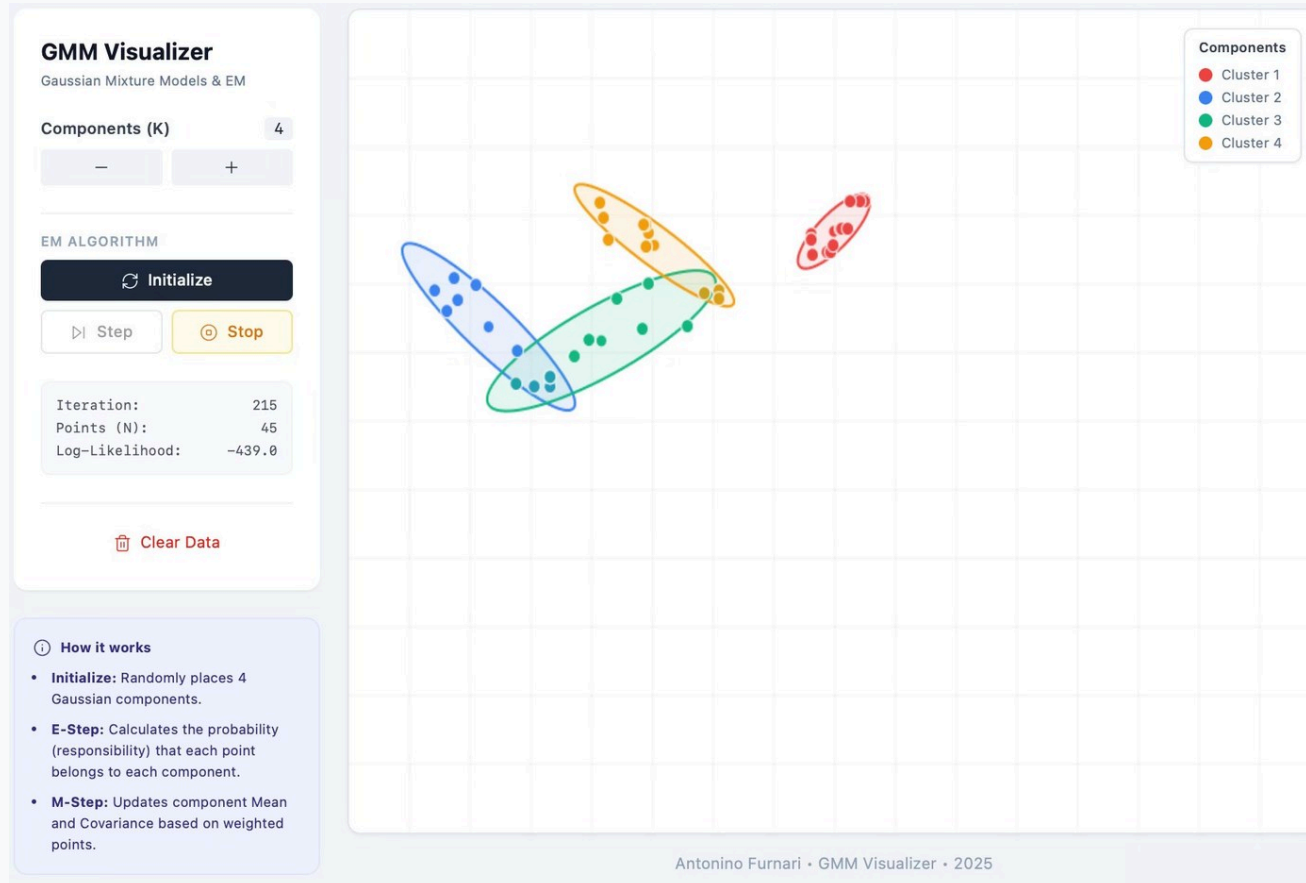
$$\log P(\mathbf{X} | \theta) = \sum_{i=1}^N \log \left[\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_i | \mu_k, \Sigma_k) \right]$$

Because of the sum **inside** the logarithm, this problem has no closed-form solution. A dedicated iterative algorithm, **Expectation-Maximization (EM)**, is used to find the parameters.

The EM algorithm is similar in spirit to K-means, but rather than just moving cluster centres, it iteratively adjusts means **and** covariance matrices — and the assignments it uses on the way are the soft responsibilities, not hard memberships.



Gaussian Mixture Models: Live Demo



The second interactive visualiser runs EM one step at a time on points you place yourself, and prints the log-likelihood at each iteration.

Open the GMM live demo

- **Initialize** places K components at random.
- The **E-step** computes the responsibility of each component for each point.
- The **M-step** updates means and covariances, weighted by those responsibilities.

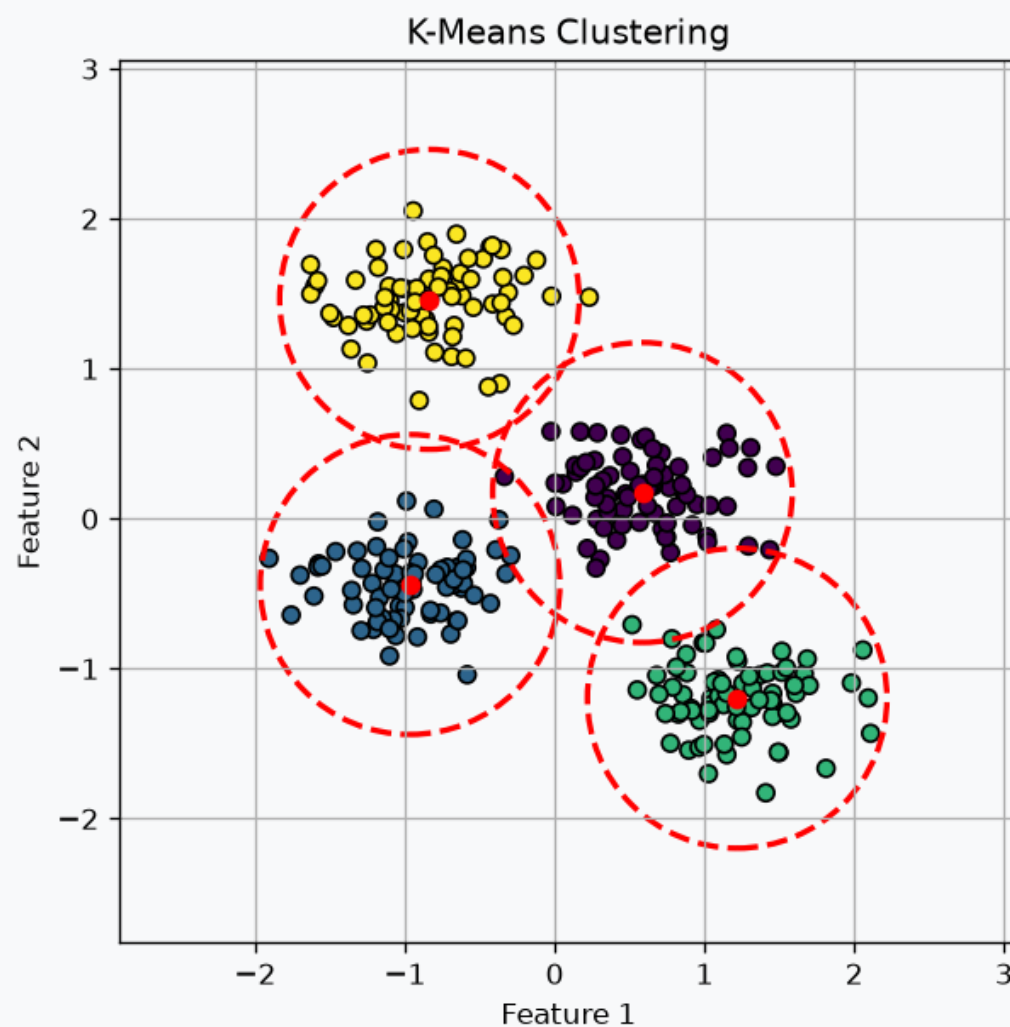
Worth watching: the log-likelihood never decreases, and the ellipses stretch and rotate in a way K-means' circles cannot.



GMM vs K-Means: A Closer Look

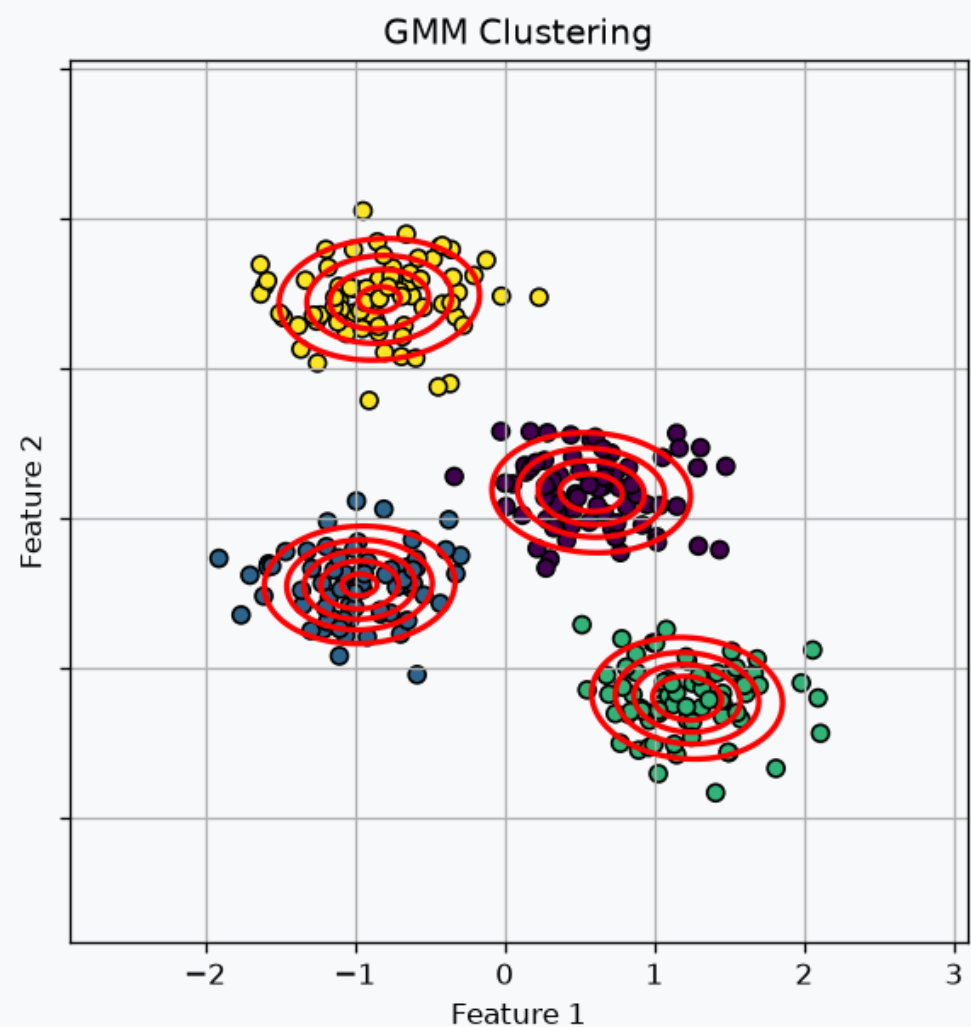
K-Means: simpler, faster

- **Hard assignment:** each point belongs to exactly one cluster, with no ambiguity.
- **Spherical, equal covariance:** clusters are assumed spherical, of the same size and axis-aligned — in effect a shared $\epsilon \mathbf{I}$ covariance.
- **Efficiency:** much faster convergence, which keeps it the popular choice when its assumptions hold.



GMM: flexible, probabilistic

- **Soft assignment:** every point has a responsibility towards each cluster.
- **Flexible covariance:** each component has its own covariance matrix, so clusters may differ in shape, size and orientation.
- **Density estimation:** beyond clustering, a GMM is a full probability density.



K-means excels in speed and simplicity when the clusters are well separated and round. GMM handles overlapping, elongated, differently-shaped groups — and the choice comes down to the structure of the data, and to whether a density model is needed at all.

Lab: Density Estimation for Anomaly Detection

We use the three density models to build an **anomaly detector** on the handwritten digits: the digit '0' is the anomaly.

The goal: the digit zero as an anomaly

- **"Normal" training data:** all images of the digits 1–9 — **1 133 images of 64 features**, a strongly multi-modal set, with nine distinct groups in it.
- **Test data:** a mix of 486 normal digits and **178 anomalous '0's**, 664 images in total.
- A good model gives a **high density** to the normal digits and a **very low** one to the zeros; the density score is then just a threshold away from a detector.

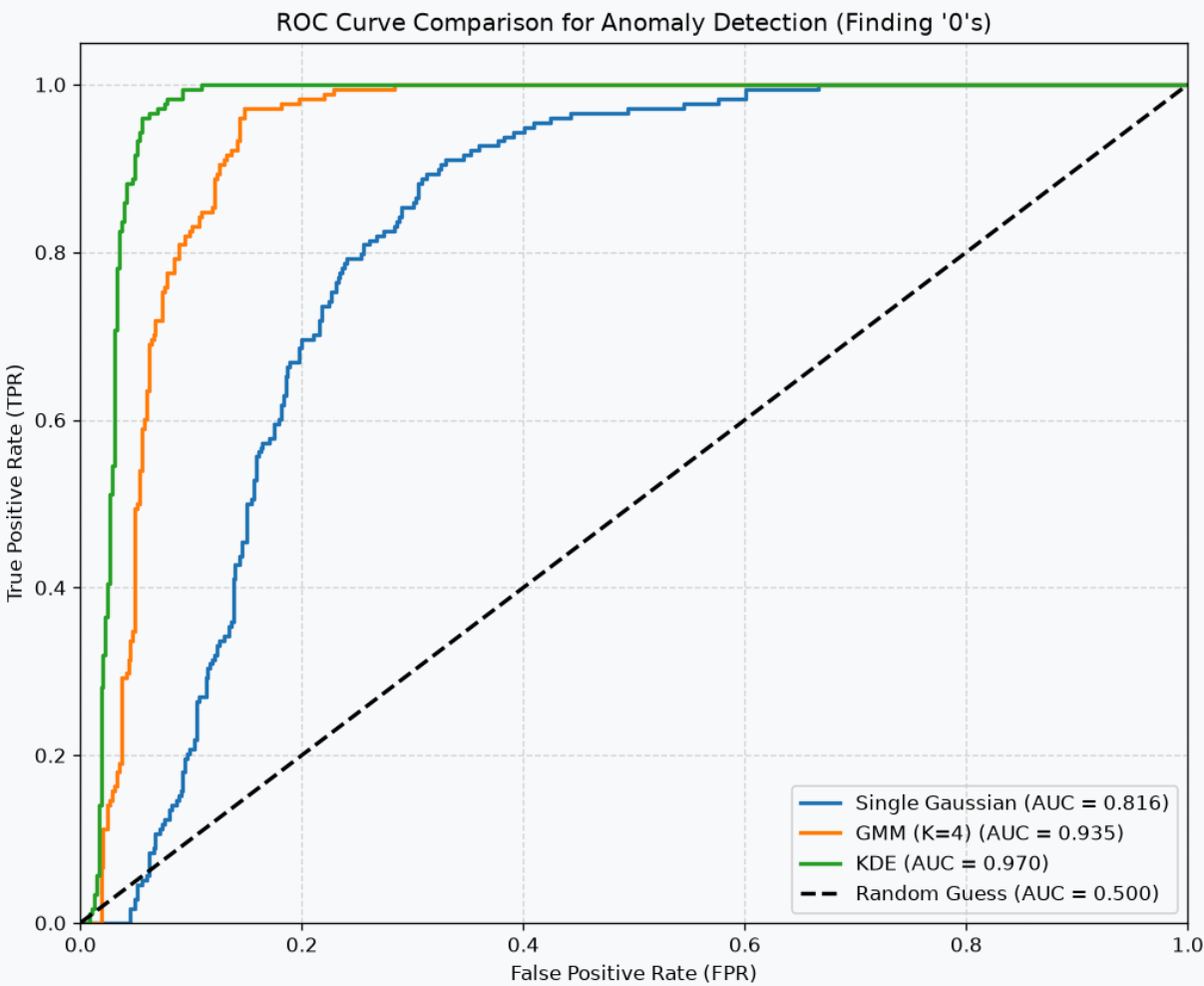
The three competitors

- **Single Gaussian:** the naive parametric model. We expect it to fail, fitting one average blob over nine different digits.
- **GMM with $K = 4$:** the smarter parametric model, with enough components to capture some of that diversity.
- **KDE:** the flexible non-parametric model, which keeps all 1 133 training images and asks them every time.



Lab: Which Model Finds the Anomalies

Step 5: quantitative comparison, the ROC curve



0.816 0.935 0.970

Single Gaussian

One blob over nine digits

GMM (K = 4)

Four components, and most of the gap closed

KDE

No assumption about shape at all

The ROC curves rank the three models exactly as the theory predicts. The single Gaussian is not useless — 0.816 is well above the diagonal — but it pays for assuming one mode where there are nine.

The price of KDE's 0.970 is that it stores the training set and evaluates a kernel against all of it for every new image. The GMM keeps four means and four covariance matrices, and gets most of the way there.



Conclusions and Next Steps



We Have Explored:

Non-parametric vs parametric density estimation

N-dimensional histograms

Kernel density estimation, and the bandwidth

Fitting Gaussians to the data

Gaussian mixture models

Soft clustering, and EM

- In the next lecture we look at principal component analysis.

References

- Multivariate KDE: en.wikipedia.org/wiki/Multivariate_kernel_density_estimation
- KDE: en.wikipedia.org/wiki/Kernel_density_estimation
- Yen-Chi Chen, *Lecture 7: Density estimation and KNN* — faculty.washington.edu/yenchic/18W_425/Lec7_knn_basis.pdf
- Section 9.2 of [1]

[1] Bishop, Christopher M. *Pattern recognition and machine learning*. Springer, 2006.

- Lecture notes: **Chapter 19 — Density estimation**

